

Базы данных

Тема 6

СУБД SQL Server 2012 – Основные элементы синтаксиса Transact-SQL.

Основные элементы синтаксиса Transact-SQL

- идентификаторы;
- типы данных;
- функции;
- выражения;
- операторы;
- комментарии;
- ключевые слова (зарезервированные);
- управляющие конструкции.

Идентификаторы

- идентификатор - имя объекта базы данных (сервера, базы данных и ее объектов - таблиц, представлений, столбцов, индексов, триггеров, процедур, ограничений и правил, и т.п.), создаваемое при определении объекта и используемое для обращения к нему;
- классы идентификаторов:
 - обычные: соответствуют правилам форматирования идентификаторов
 - первый символ – буква или символы `_`, `@` или `#` ;
 - последующие символы (до 128) – буквы, десятичные цифры или символы `_`, `@`, `#` или `$` ;
 - не должны совпадать с зарезервированными словами (независимо от регистра);
 - не должны содержать пробелы и спецсимволы;
 - расширенные (идентификаторы с разделителем):
 - могут содержать любые символы;
 - заключаются в квадратные скобки или двойные кавычки: `[..]`, `".."`.

Виды идентификаторов

- ИМЕНА ОБЪЕКТОВ:

`[[[server.][database.][schema_name].]object_name`

`server.database.schema_name.object_name`

`server.database..object_name`

`server..schema_name.object_name`

`server...object_name`

`database.schema_name.object_name`

`database..object_name`

`schema_name.object_name`

`database_name.schema_name.object_name.column_name`

`database_name..object_name.column_name`

`schema_name.object_name.column_name`

`object_name.column_name`

- локальные переменные и параметры:

`@.....`

- локальные временные таблицы и процедуры:

`#.....`

- глобальные временные объекты (таблицы и процедуры):

`##.....`

Видимость объектов и правила квалификации

- при создании объекта Microsoft SQL Server 2005 использует следующие значения по умолчанию для неуказанных частей имени
- при обращении к объекту SQL Server использует следующие значения по умолчанию для неуказанных частей имени

Неуказанная часть	Значение по умолчанию
<i>Сервер</i>	сервером по умолчанию является локальный сервер.
<i>База данных</i>	базой данных по умолчанию является текущая база данных.
<i>Имя схемы</i>	именем схемы по умолчанию является имя пользователя в указанной базе данных, связанное с идентификатором имени входа текущего соединения.

Неуказанная часть	Значение по умолчанию
<i>Сервер</i>	сервером по умолчанию является локальный сервер.
<i>База данных</i>	базой данных по умолчанию является текущая база данных.
<i>Имя схемы</i>	именем схемы по умолчанию является имя пользователя в указанной базе данных, связанное с идентификатором имени входа текущего соединения. Если этот пользователь не владеет объектом с указанным именем, SQL Server ищет объект с указанным именем, принадлежащий владельцу базы данных (dbo).

Идентификаторы с разделителями

```
SET QUOTED_IDENTIFIER ON
```

```
GO
```

```
CREATE TABLE "table" (  
    tablename char(128) NOT NULL,  
    "USER" char(128) NOT NULL,  
    "SELECT" char(128) NOT NULL,  
    "INSERT" char(128) NOT NULL,  
    "UPDATE" char(128) NOT NULL,  
    "DELETE" char(128) NOT NULL
```

```
);
```

```
GO
```

```
SET QUOTED_IDENTIFIER ON;
```

```
GO
```

```
SELECT * FROM "table";
```

```
GO
```

```
SET QUOTED_IDENTIFIER OFF;
```

```
GO
```

```
SELECT * FROM [table];
```

```
GO
```

```
EXEC sp_help 'dbo.[tab.one]'
```

```
SET QUOTED_IDENTIFIER ON
```

```
GO
```

```
EXEC sp_help 'dbo."tab.one"'
```

```
GO
```

Числовые константы

Использование констант	Пример
битовые константы (bit)	0 или 1
целочисленные константы (Integer)	1894 -2 0x3AF4 0xABC
константы decimal	1894.1204 2.0
вещественные константы (float и real)	-12.34 101.5E5 0.5E-2
финансовые константы (money)	\$12 \$542023.14
константы uniqueidentifier	0xff19966f868b11d0b42d00c04fc964ff '6F9619FF-8B86-D011-B42D-00C04FC964FF'
константы двоичных строк	0x12Ef 0x69048AEFDD010E

Строковые константы

- заключены в одинарные кавычки:
`'Character string'`
- строки Unicode используют префикс N:
`N'Unicode string'`
- одинарные кавычки, содержащиеся в строке, дублируются:
`'O' 'Brien'`
- если установлено значение `SET QUOTED_IDENTIFIER OFF`, то можно использовать двойные кавычки:
`"string"`

Использование констант	Пример
символьные строки	'O"Brien' 'The level for job_id: %d should be between %d and %d.'
строки в Юникоде	N'Michl'

Константы для даты

- формат ISO 8601 (не зависит от настроек формата даты и языка):

`'yyyy-mm-ddThh:mm:ss [.mmm] '`

`'2004-05-23T14:25:10 '`

`'2004-05-23T14:25:10.487 '`

- алфавитный формат:

`'20 Apr 2005 '`

`'1996 APR[IL] [15] '`

- числовой формат:

- используется значение SET DATEFORMAT
- используются разделители '/', '-' или '.'

`'20/4/2005 '`

`'04/15/98 '`

`'04:24 PM '`

`'14:30:24 '`

- строковый формат без разделителей (6- и 8-разрядные строки):

- год, месяц, число без разделителей;
- месяц и число из двух цифр;

`'20050420 '`

`'960420 '`

Функции

- функции можно использовать и включать в следующие конструкции:
 - список выбора инструкции SELECT;
 - предложение WHERE инструкций SELECT, INSERT, DELETE или UPDATE для ограничения количества строк, удовлетворяющих условиям запроса;
 - условие поиска в предложении WHERE представления, которое должно динамически соответствовать пользователю или среде во время выполнения;
 - любое выражение;
 - ограничение CHECK или триггер для поиска указанных значений при вставке данных;
 - ограничение DEFAULT или триггер для вставки значений по умолчанию;

Категории функций

Категория функции	Описание
Статистические функции	Выполняют операции, объединяющие несколько значений в одно.
Функции настройки	Скалярные функции, возвращающие сведения о параметрах конфигурации.
Криптографические функции	Поддерживают шифрование, дешифрование, цифровые подписи и их проверку.
Функции для работы с курсорами	Возвращают сведения о состоянии курсора.
Функции даты и времени	Изменяют значения даты и времени.
Математические функции	Служат для выполнения тригонометрических, геометрических и других математических операций.
Функции метаданных	Возвращают сведения об атрибутах баз данных и объектов баз данных.
Ранжирующая функция	Недетерминированные функции, возвращающие ранжирующее значение для каждой строки секции.
Функции наборов строк	Возвращают результирующие наборы, которые можно использовать вместо ссылок на таблицу в инструкции Transact-SQL.
Функции безопасности	Возвращают сведения о пользователях и ролях.
Строковые функции	Изменяют значения char , varchar , nchar , nvarchar , binary и varbinary .
Системные функции	Работают или возвращают отчет о различных параметрах и объектах уровня системы.
Системные статистические функции	Возвращают сведения о производительности SQL Server.
Функции обработки текста и изображений	Изменяют значения text и image .

Категории функций: примеры

Категория функции	Описание
Статистические функции	AVG, MIN, CHECKSUM, SUM, CHECKSUM_AGG, STDEV, COUNT, STDEVP, COUNT_BIG, VAR, GROUPING, VARP, MAX
Функции конфигурации	@@OPTIONS, @@SERVERNAME, @@LANGUAGE, @@SERVICENAME, @@LOCK_TIMEOUT @@MAX_CONNECTIONS @@MAX_PRECISION @@VERSION @@NESTLEVEL,
Криптографические функции	SignByAsymKey, VerifySignedByAsmKey, SignByCert, ...
Функции для работы с курсорами	@@CURSOR_ROWS, CURSOR_STATUS, @@FETCH_STATUS
Функции даты и времени	DATEADD, DATEDIFF, DATENAME, DATEPART, DAY, GETDATE, GETUTCDATE, MONTH, YEAR
Математические функции	ABS, RAND, COS, EXP, ROUND, FLOOR, SIGN, ATAN, LOG, SIN
Функции метаданных	DB_NAME, OBJECT_NAME, DATABASEPROPERTY, FILEPROPERTY, ...
Ранжирующая функция	RANK, NTILE, DENSE_RANK, ROW_NUMBER
Функции наборов строк	CONTAINSTABLE, OPENQUERY, FREETEXTTABLE, OPENROWSET, OPENDATASOURCE, OPENXML
Функции безопасности	CURRENT_USER, SCHEMA_ID, USER_NAME, SUSER_ID, IS_MEMBER
Строковые функции	SOUNDEX, STR, REPLACE, LEFT, SUBSTRING, LEN, LOWER, LTRIM, ...
Системные функции	ISDATE, ISNULL, ROWCOUNT, NEWID, HOST_ID, CAST, CONVERT, ...
Системные статистические функции	@@CONNECTIONS, @@TOTAL_WRITE, @@CPU_BUSY, @@PACK_SENT, @@TIMETICKS, @@TOTAL_ERRORS, @@IO_BUSY, @@TOTAL_READ, ...
Функции обработки текста и изображений	PATINDEX TEXTVALID TEXTPTR

Выражения

- выражение - это сочетание идентификаторов, значений и операторов, которое SQL Server 2012 может вычислить для получения результата;
- выражения могут использоваться в различных контекстах во время доступа к данным или их изменения, например:
 - как часть данных, получаемых в запросе;
 - в качестве условий поиска данных, отвечающих определенному набору критериев;
- выражение может быть:
 - константой;
 - функцией;
 - именем столбца;
 - переменной;
 - вложенным запросом;
 - функцией CASE, NULLIF или COALESCE;
 - выражение также может быть построено из комбинаций этих сущностей, соединенных операторами.

Категории операторов

Операции	Категория оператора
Сравнение значения с другим значением или выражением.	Операторы сравнения
Проверка истинности условия, такого как AND, OR, NOT, LIKE, ANY, ALL, IN.	Логические
Сложение, вычитание, умножение, деление и взятие остатка от деления.	Арифметические операторы
Выполнение действия над одним операндом, например положительным или отрицательным, или над дополнением.	Унарные
Временное обращение регулярных числовых значений в целочисленные и выполнение побитовой арифметической операции.	Битовые операторы
Постоянное или временное объединение двух строк (символьных или двоичных) в одну строку.	Оператор сцепления строк
Присваивание значения переменной или связывание столбца результирующего набора с псевдонимом.	Присвоение

Комментарии

- комментарии представляют собой неисполняемые текстовые строки в программном коде;
- комментарий должен целиком содержаться внутри пакета;
- SQL Server 2012 поддерживает два типа символов выделения комментариев:
 1. -- (двойной дефис)
 - символы комментария могут находиться в той же строке, что и исполняемый программный код, или в отдельных строках;
 - все, что расположено после двойного дефиса и до конца строки, является частью комментария;
 - в многострочных комментариях двойной дефис необходимо указывать в начале каждой строки комментария;
 2. /* ... */
 - символы комментария могут находиться в той же строке, что и исполняемый программный код, или в отдельных строках, или даже внутри исполняемого кода;
 - все, что расположено после комбинации символов, открывающей комментарий (/*), и до комбинации символов, закрывающей комментарий (*/), рассматривается как часть комментария.

Пакеты

- пакет - это группа из одной или более инструкций языка Transact-SQL, отправленных единовременно от приложения к SQL Server 2012 для выполнения. SQL Server компилирует инструкции пакета в единый исполняемый модуль, называемый планом выполнения, который затем последовательно выполняется;
- ошибка компиляции, например синтаксическая ошибка, прекращает компиляцию плана выполнения, поэтому никакие инструкции в пакете не выполняются;
- ошибка выполнения, например арифметическое переполнение или нарушение ограничения, приведет к одному из двух результатов:
 - большинство ошибок времени выполнения останавливают текущую инструкцию и инструкции, следующие за ней в пакете;
 - некоторые ошибки времени выполнения, например нарушения ограничений, останавливает только текущую инструкцию, а все остальные инструкции в пакете будут выполнены;
- ошибка выполнения не окажет влияния на инструкции, выполненные до инструкции, которая столкнулась с ошибкой времени выполнения. (единственное исключение - если пакет находится в транзакции, и из-за ошибки будет выполнен откат транзакции, - в этом случае будет выполнен откат любых незафиксированных изменений данных, сделанных перед ошибкой времени выполнения).

Задание пакетов

- наряду с хранимыми процедурами, триггерами и сценариями, пакеты являются механизмом реализации процессов, которые не могут быть реализованы с использованием одиночной инструкции Transact-SQL;
- пакеты реализованы как часть API базы данных:
 - в ADO пакет представляет собой строку инструкций Transact-SQL, заключенную в свойстве **CommandText** объекта **Command**

```
Dim Cmd As New ADODB.Command
Set Cmd.ActiveConnection = Cn
Cmd.CommandText = "SELECT * FROM Purchasing.Vendor; SELECT
                  * FROM Production.Product"

Cmd.CommandType = adCmdText
Cmd.Execute
```

- В OLE DB пакет - это строка инструкций Transact-SQL, заключенная в строку, которая используется для задания текста команды;
 - В ODBC пакет представляет собой строку инструкций Transact-SQL, заключенную в вызове **SQLPrepare** или **SQLExecDirect**;
- SQL Server Management Studio и другие программы используют команду GO для обозначения конца пакета. GO - это не инструкция Transact-SQL; она говорит программам, сколько инструкций SQL должно быть включено в пакет.

Объявление переменных

- инструкция DECLARE инициализирует переменную Transact-SQL следующим образом:
 - задает имя, начинающееся с одного знака @;
 - задает тип данных, определенный системой или пользователем, и длину; для числовых переменных указывается масштаб и точность;
 - присваивает созданной переменной значение NULL
- инструкция DECLARE позволяет объявить несколько переменных одинакового или разного типов через запятую;
- областью видимости переменной называют диапазон инструкций Transact-SQL, которые могут к ней обращаться;
- областью видимости переменной являются все инструкции между ее объявлением и концом пакета или хранимой процедуры, где она объявлена;
- для присвоения значения переменной можно:
 - применить инструкцию SET (предпочтительный способ);
 - указать переменную в списке выбора инструкции SELECT;

```
-- Declare two variables.  
DECLARE @FirstNameVariable nvarchar(20) ,  
        @RegionVariable nvarchar(30)  
-- Set their values.  
SET @FirstNameVariable = N'Anne'
```

Управление выполнением

- язык Transact-SQL содержит специальные ключевые слова, известные как язык управления выполнением, которые управляют порядком выполнения инструкций на языке Transact-SQL, блоками инструкций и хранимыми процедурами;
- инструкции управления выполнением не могут быть распределены по разным пакетам или хранимым процедурам;
- существуют следующие ключевые слова, управляющие выполнением:
 - **BEGIN . . . END**
 - **BREAK**
 - **GOTO**
 - **CONTINUE**
 - **IF . . . ELSE**
 - **WHILE**
 - **RETURN**
 - **WAITFOR**

Составной оператор BEGIN...END

- инструкции BEGIN и END используются для группирования нескольких инструкций Transact-SQL в одном логическом блоке;
- инструкции BEGIN и END используются, когда:
 - в цикл WHILE необходимо включить блок инструкций;
 - в элемент функции CASE необходимо включить блок инструкций;
 - в предложение IF или ELSE необходимо включить блок инструкций

```
IF (@@ERROR <> 0)
BEGIN
    SET @ErrorSaveVariable = @@ERROR
    PRINT 'Error encountered, ' +
        CAST(@ErrorSaveVariable AS VARCHAR(10))
END
```

Ветвления и переходы

- ветвление (проверка выполнения условия):

IF *expression*
statement

ELSE
statement

- безусловное завершение запроса, хранимой процедуры или пакета:

RETURN [*@ReturnCode*]

- безусловный переход к метке:

GOTO *label*

...

label_name:

...

Организация цикла

- ЦИКЛ:

WHILE *expression*
statement

- управление циклом:

- завершение цикла **BREAK**
- переход к очередной итерации **CONTINUE**

```
DECLARE abc CURSOR FOR
SELECT * FROM Purchasing.ShipMethod;
OPEN abc;
FETCH NEXT FROM abc
WHILE (@@FETCH_STATUS = 0)
    FETCH NEXT FROM abc;
CLOSE abc;
DEALLOCATE abc;
GO
```

Инструкция WAITFOR

- инструкция WAITFOR приостанавливает выполнение пакета, хранимой процедуры или транзакции до тех пор, пока:
 - не пройдет заданный интервал времени:
`DELAY(time_to_pass)`
 - не наступит заданное время:
`TIME(time_to_execute)`
 - инструкция RECEIVE не изменит или не возвратит, по крайней мере одну строку в очередь компонента Service Broker:
`RECEIVE / TIMEOUT`

```
WAITFOR DELAY '00:00:02'
```

```
SELECT EmployeeID FROM MyDB.HumanResources.Employee
```

Функция CASE

- функция CASE - специальное выражение Transact-SQL, которое позволяет формировать альтернативные значения в зависимости от условия;
- функция CASE состоит из:
 - ключевого слова CASE;
 - имени преобразуемого столбца;
 - предложений WHEN, которые задают выражения для поиска, и предложений THEN, которые задают выражения для замены;
 - ключевого слова END;
 - необязательного предложения AS, задающего псевдоним функции CASE.

```
SELECT ProductNumber, Name, 'Price Range' = CASE
WHEN ListPrice = 0 THEN 'Mfg item - not for resale'
WHEN ListPrice < 50 THEN 'Under $50'
WHEN ListPrice >= 50 and ListPrice < 250
                                THEN 'Under $250'
WHEN ListPrice >= 250 and ListPrice < 1000
                                THEN 'Under $1000'
ELSE 'Over $1000'
END [AS...]
FROM Production.Product
ORDER BY ProductNumber ;
GO
```


Обработка ошибок

- ошибки, вызываемые компонентом Database Engine, можно обрабатывать на двух уровнях:
 - в компоненте Database Engine, вводя код обработки ошибок в пакеты Transact-SQL, хранимые процедуры, триггеры или в пользовательские функции. Механизмы обработки ошибок Transact-SQL включают:
 - конструкцию TRY...CATCH...END CATCH (получение информации об ошибке с помощью функций ERROR_LINE, ERROR_MESSAGE, ERROR_NUMBER, ERROR_PROCEDURE, ERROR_SEVERITY и ERROR_STATE)
 - инструкции RAISERROR / THROW / PRINT (возврат приложению сообщения в формате системных ошибок или предупреждений / в формате символьной строки)
 - функцию @@ERROR (получение номера ошибки, созданной предыдущей инструкцией языка Transact-SQL)
 - на уровне вызывающего приложения. Каждый из API, используемых приложением для доступа к компоненту Database Engine, наделен механизмами для возвращения сведений об ошибках соответствующему приложению.

Вопросы к экзамену

- Основные элементы синтаксиса Transact-SQL: именование объектов, идентификаторы, константы, встроенные функции, выражения.
- Основные элементы синтаксиса Transact-SQL: управление выполнением, обработка ошибок